

Low-Discrepancy Sampling on Higher-Dimensional Spheres

Wai-Shing Luk

September 25, 2026

Abstract

This paper studies the generation of low-discrepancy point sets on n -dimensional spheres. Low-discrepancy sequences (LDS) are widely used in numerical integration, optimisation and simulation, and the quality of a point set on S^n is governed by three properties: uniformity, determinism and incrementality. We propose a construction of low-discrepancy sequences on S^n based on the van der Corput sequence, and describe the algorithm and its implementation in detail. Numerical experiments compare the proposed method with random sampling and with established approaches, namely the Hopf-coordinate and cylindrical-mapping methods.

1 Motivation

Low-discrepancy sequences (LDS) arise throughout mathematics, computer science and engineering. Compared with random sampling, they distribute points more evenly over the sampling domain, which makes them valuable for numerical integration, optimisation and simulation. Sampling the three-dimensional sphere is well understood, but efficient methods for higher dimensions remain scarce.

Low-discrepancy sequences offer three advantages over random sampling:

1. **Uniformity.** The points are spread evenly over the sampling region, avoiding clusters and large gaps.
2. **Determinism.** For a fixed starting point and parameters, the sequence is exactly reproducible.
3. **Incrementality.** This is the most important property: points can be appended one at a time

while the whole set stays evenly distributed. It matters when the required number of samples is not known in advance, for example in robotics, where one may stop after a few points or change strategy.

The main drawback is cost: generation becomes slower as the number of points grows.

Higher dimensions introduce further difficulties. An even distribution becomes harder to achieve as the dimension increases, an effect known as the *curse of dimensionality*, and, while methods for the 2-sphere are well established, the higher-dimensional theory is still developing. In particular, no single strategy is known that applies uniformly across all dimensions while preserving uniformity, determinism and incrementality.

This paper proposes a construction of low-discrepancy sequences on S^n based on the van der Corput sequence, addressing the difficulties of high-dimensional sampling while retaining the properties above. The method is relevant to several areas:

- **Robot motion planning** (Yershova et al. 2010). On S^3 and $SO(3)$, Halton point sets are evenly distributed and therefore suit path planning and attitude control, improving both computational efficiency and trajectory accuracy.
- **Wireless communication coding** (Utkovski and Lindner 2006). In spherical coding for MIMO systems the points serve as codewords, improving signal stability and transmission quality.
- **Multivariate empirical mode decomposition** (Rehman and Mandic 2010). Halton points support more accurate signal models.
- **Filter bank design** (Mandic et al. 2011). They



Figure 1: Example of the van der Corput sequence

allow more precise filter parameters to be constructed.

- **Statistical and machine learning.** They provide deterministic, evenly distributed coverage of a normalised parameter space.

Section 2 reviews low-discrepancy sequences, Section 3 presents the proposed method on S^n , Section 4 reports the numerical experiments, and Section 5 concludes.

2 Overview of Low-Discrepancy Sequences

2.1 The van der Corput Sequence

The van der Corput sequence is the one-dimensional low-discrepancy sequence on $[0, 1]$. It is constructed by reversing the base- b digits of the non-negative integers, where b is usually prime, and it is named after the Dutch mathematician Johannes van der Corput, who introduced it in 1935. Writing $n = \sum_{k \geq 0} a_k b^k$ with digits $a_k \in \{0, \dots, b-1\}$, the n -th term is the *radical inverse*

$$\phi_b(n) = \sum_{k \geq 0} \frac{a_k}{b^{k+1}},$$

that is, the base- b digits of n reflected about the radix point.

Incrementality is visible in the figure: the first ten points (orange) and the next ten (purple) are each evenly spread, and their union is evenly spread as well. Few other sampling methods have this property.

Concretely, given an index k and a base b (default 2), the generator returns the k -th value of the sequence in $[0, 1]$. It repeatedly divides k by b and assembles the fractional value from the remainders; equivalently, it writes k in base b and reads the digits in reverse after the point. For instance, with $b = 2$ the third element is 011_2 , which reverses to $0.11_2 = 0.75$.

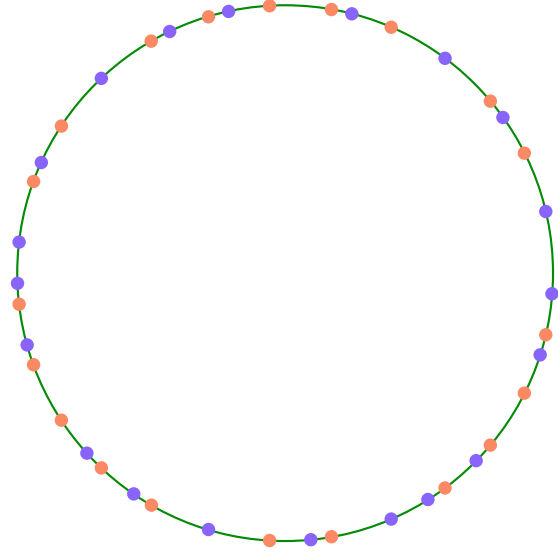


Figure 2: Example of the circle sequence

2.2 Unit Circle S^1

The construction extends to the unit circle by treating the van der Corput value as an angle:

1. take the next value from the van der Corput sequence;
2. multiply it by 2π to obtain an angle $\theta \in [0, 2\pi)$;
3. return $[x, y] = [\cos \theta, \sin \theta]$.

The essential step is the conversion of a one-dimensional sequence into points on a two-dimensional circle by interpreting the sequence value as an angle.

2.3 Halton Sequence on $[0, 1]^n$

The Halton sequence combines two or more van der Corput sequences that use distinct prime bases; it is named after Halton and Rutishauser, who developed it in the 1960s. The resulting points are evenly distributed over the unit square, without the regularity of a grid, which makes the sequence useful for two-dimensional sampling.

The construction generalises to higher dimensions: combining n van der Corput sequences with pairwise-coprime bases gives

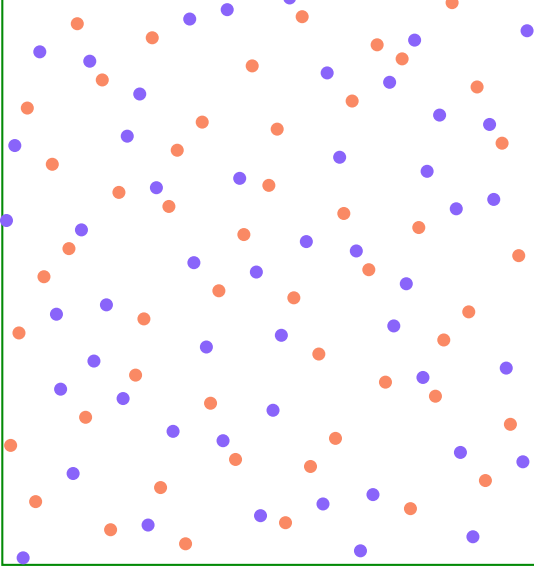


Figure 3: Example of the Halton sequence

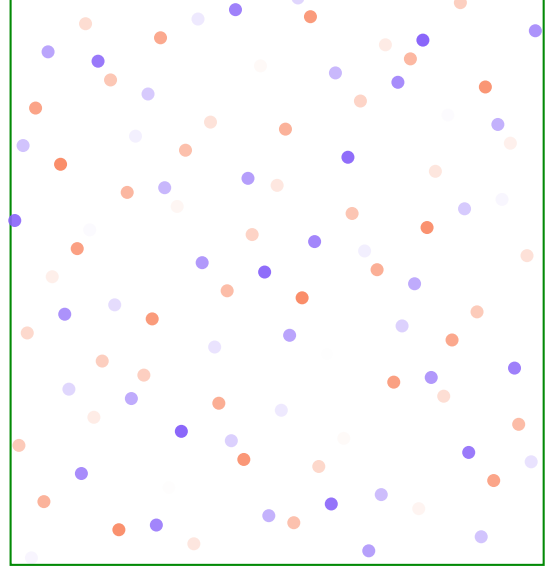


Figure 4: Example of the Halton sequence in 3D

$$[x_1, x_2, \dots, x_n] = [\text{vdc}(k, b_1), \text{vdc}(k, b_2), \dots, \text{vdc}(k, b_n)].$$

Such point sets are the basis of quasi-Monte Carlo (QMC) methods.

2.4 Unit Sphere S^2

Points on the unit sphere can be generated by combining a one-dimensional sequence for the height with a circular sequence for the horizontal position; this cylindrical mapping has been used in computer graphics (Wong et al. 1997). Given the azimuth φ and the height z , the point is

$$[x, y, z] = [r \cos \varphi, r \sin \varphi, z], \quad r = \sqrt{1 - z^2},$$

where

- $z = 2 \text{vdc}(k, b_2) - 1 \in [-1, 1]$, and
- $\varphi = 2\pi \text{vdc}(k, b_1) \in [0, 2\pi)$.

2.5 S^3 and $SO(3)$

The construction extends to the 3-sphere (a four-dimensional sphere) through the Hopf fibration (Mitchell 2008; Yershova et al. 2010), which was originally used to build optimal deterministic grids on S^3 and $SO(3)$. Hopf coordinates are

- $x_1 = \cos(\theta/2) \cos(\psi/2)$,
- $x_2 = \cos(\theta/2) \sin(\psi/2)$,
- $x_3 = \sin(\theta/2) \cos(\varphi + \psi/2)$,
- $x_4 = \sin(\theta/2) \sin(\varphi + \psi/2)$,

and S^3 is a principal circle bundle over S^2 . The cylindrical mapping of the previous subsection does not extend to higher dimensions, which is why the Hopf construction is needed.

Three van der Corput sequences supply the angles:

- $\varphi = 2\pi \text{vdc}(k, b_1)$;
- $\psi = 2\pi \text{vdc}(k, b_2)$ for $SO(3)$, or $\psi = 4\pi \text{vdc}(k, b_2)$ for S^3 ;
- $z = 2 \text{vdc}(k, b_3) - 1$, so that $\theta = \cos^{-1} z$.

The first two values are scaled to angles, the third determines θ , and the four coordinates follow from

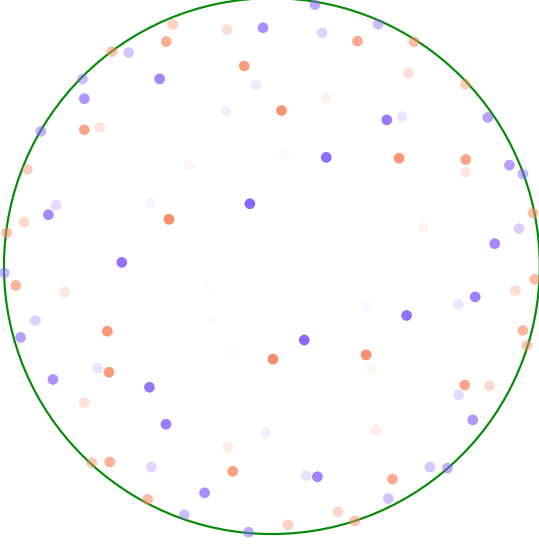


Figure 5: Example of the sphere sequence

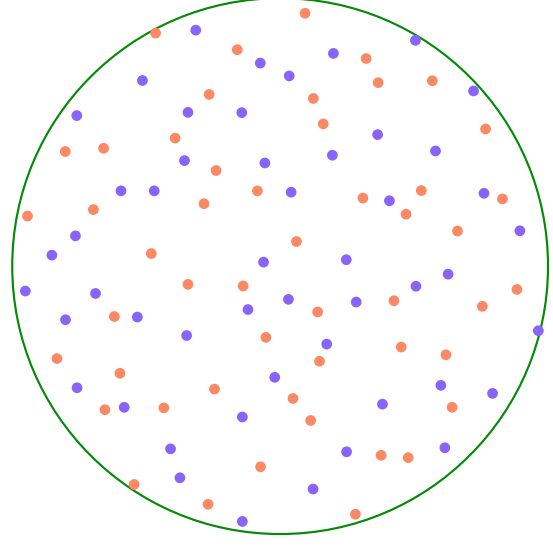


Figure 6: Example of the unit-disk sequence

the Hopf formulae.

3 Our Approach

3.1 Uniform Sampling on a Unit Disk

The unit disk illustrates the general principle. To sample it uniformly one examines the surface element, which in polar coordinates (r, θ) is

$$dA = r dr d\theta. \quad (1)$$

The factor r means that a distribution uniform in θ alone would over-sample the centre. Integrating the radial part,

$$\int r dr = \frac{1}{2} r^2, \quad (2)$$

shows that the radial weight is r , so the inverse function needed to cancel it is the square root. The disk is therefore sampled uniformly by

- $\theta = 2\pi \text{vdc}(k, b_1)$,
- $r = \sqrt{\text{vdc}(k, b_2)}$,

- $[x, y] = [r \cos \theta, r \sin \theta]$.

The same reasoning applies to the boundary and to the interior.

3.2 Higher Dimensions: S^3 and Beyond

In higher dimensions the surface element is more complicated and its inverse is no longer obvious. The polar coordinates of S^3 are

- $x_0 = \cos \theta_3$,
- $x_1 = \sin \theta_3 \cos \theta_2$,
- $x_2 = \sin \theta_3 \sin \theta_2 \cos \theta_1$,
- $x_3 = \sin \theta_3 \sin \theta_2 \sin \theta_1$,

with surface element

$$dA = \sin^2(\theta_3) \sin(\theta_2) d\theta_1 d\theta_2 d\theta_3.$$

The construction proceeds recursively:

- start from $p_0 = [\cos \theta_0, \sin \theta_0]$ with $\theta_0 = 2\pi \text{vdc}(k, b_0)$;
- with $f_2(\theta) = \int \sin^2 \theta d\theta = \frac{1}{2}(\theta - \cos \theta \sin \theta)$, map $\text{vdc}(k, b_2)$ onto f_2 by $t_2 = (\pi/2) \text{vdc}(k, b_2)$;

- recover $\theta_2 = f_2^{-1}(t_2)$ by table lookup;
- set $p_2 = [\sin \theta_2 \cdot p_1, \cos \theta_1]$.

3.3 Generalisation to S^n

The same recursion applies in any dimension, but its inverse function has no closed form for $n \geq 2$. Only the cases $n = 0$ (a point) and $n = 1$ (the circle) admit closed-form inverses; for $n \geq 2$ the inverse must be tabulated numerically. The mapping function is defined recursively, so higher-dimensional spheres are built from lookup tables.

Two classes implement the idea: **Sphere3** generates points on S^3 directly, and **SphereN** builds higher-dimensional spheres recursively, each instance handling one dimension and delegating the rest to a lower-dimensional generator. Because the recursion bottoms out at S^3 , points can be generated in any dimension, limited only by memory.

The polar coordinates of S^n are

- $x_0 = \cos \theta_n$,
- $x_1 = \sin \theta_n \cos \theta_{n-1}$,
- $x_2 = \sin \theta_n \sin \theta_{n-1} \cos \theta_{n-2}$,
- $\dots$
- $x_{n-1} = \sin \theta_n \sin \theta_{n-1} \dots \cos \theta_1$,
- $x_n = \sin \theta_n \sin \theta_{n-1} \dots \sin \theta_1$,

with surface element

$$d^n A = \sin^{n-2}(\theta_{n-1}) \sin^{n-1}(\theta_{n-2}) \dots \sin(\theta_2) d\theta_1 d\theta_2 \dots d\theta_{n-1}.$$

3.4 How to Generate the Point Set

A point set on S^n is generated as follows:

- set $p_0 = [\cos \theta_1, \sin \theta_1]$ with $\theta_1 = 2\pi \text{vdc}(k, b_1)$;
- let $f_j(\theta) = \int \sin^j \theta d\theta$ on $(0, \pi)$, defined recursively by

$$f_j(\theta) = \begin{cases} \theta & j = 0, \\ -\cos \theta & j = 1, \\ (1/n)(-\cos \theta \sin^{j-1} \theta & j \geq 2. \\ \quad + (n-1)f_{j-2}(\theta)) \end{cases}$$

}

For example, the two forms of f_2 are

$$\begin{aligned} (1/3)(-\cos \theta \sin^2 \theta - 2 \cos \theta) \\ (-1/3) \cos \theta (3 - \cos^2 \theta). \end{aligned}$$

Note that f_j is monotone increasing on $(0, \pi)$;

- map $\text{vdc}(k, b_j)$ uniformly onto f_j by $t_j = f_j(0) + (f_j(\pi) - f_j(0)) \text{vdc}(k, b_j)$;
- set $\theta_j = f_j^{-1}(t_j)$ by table lookup;
- assemble the point recursively as $p_n = [\cos \theta_n, \sin \theta_n \cdot p_{n-1}]$.

3.5 Implementation

The implementation follows the recursion directly. Its components are:

1. the van der Corput generator, which produces uniform values in $[0, 1]$;
2. interpolation routines that map these values onto the sphere;
3. the abstract **SphereGen** interface, which fixes the common methods **pop** and **reseed**;
4. the recursive generators **Sphere3** and **SphereN**.

Generation begins by constructing a **SphereN** object, which uses **Sphere3** or further **SphereN** instances for the lower dimensions. Each point combines one van der Corput value, through sine, cosine and interpolation, with the coordinates of the lower-dimensional sphere.

3.6 Software and Cross-Language Verification

Reference implementations of the construction are available in several languages: **lds-gen** (Python), **lds-rs** (Rust), and the C++20 libraries **lds-cpp** and **lds-gen-cpp**. All four provide the same core generators (**VdCorput**, **Halton**, **Circle**, **Disk**, **Sphere**, **Sphere3Hopf** and **HaltonN**), whereas the recursive **Sphere3** and **SphereN** generators currently exist in Python, Rust and **lds-gen-cpp** only.

The implementations agree numerically. For a fixed seed the core generators produce bit-identical output across languages, and the recursive sphere generators

agree to at least fifteen decimal places; the only discrepancy observed was a single coordinate differing in the sixteenth decimal place, which is consistent with machine epsilon for double precision and with the different `libm` implementations used by the compilers and interpreters.

Because each generator is stateful, concurrency is handled differently. Python guards `pop` and `reseed` with a lock, Rust uses a lock-free atomic counter, and the C++ implementations are not thread-safe and assume a single thread. This choice interacts with performance: measured timings for the recursive sphere generators, in nanoseconds per point, are 657 (C++), 1035 (Rust) and 7749 (Python) for **Sphere3**; 1665, 1607 and 12153 for **SphereN** on S^4 ; and 2312, 2020 and 48443 for **SphereN** on S^5 . Compile-time template bases let the C++ compiler replace the modulo by a multiplication, making it fastest for the shallow generator, while Rust’s lock-free counter scales better with recursion depth and is fastest for S^4 and S^5 ; Python carries interpreter overhead in the table lookup and the trigonometric evaluation.

4 Numerical Experiments

The experiments compare the quality of point distributions obtained by random sampling and by low-discrepancy sequences. For each method the points are generated, their convex hull is built with `scipy.spatial.ConvexHull`, and a dispersion measure is computed from the hull’s simplices. Dispersion is the spread of neighbour distances,

$$\max_{a \in \mathcal{N}(b)} \{D(a, b)\} - \min_{a \in \mathcal{N}(b)} \{D(a, b)\},$$

where $D(a, b) = \sqrt{1 - a^\top b}$. A smaller value indicates a more uniform point set.

The parameters are fixed: 600 points, on a five-dimensional sphere for the random method and a four-dimensional sphere for the LDS methods. The dispersion of each method is compared with its expected value.

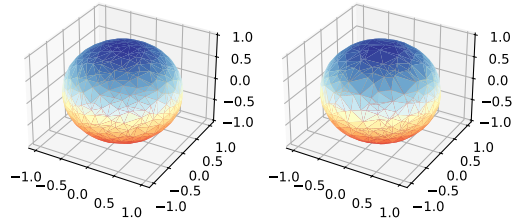


Figure 7: Left: our method, right: random

Random points on S^n are obtained by normalising a vector drawn from a multidimensional Gaussian density; the spherical symmetry of the density makes the result uniformly distributed on S^n (Fishman 1996). The LDS points are produced by the **SphereN** and **CylindN** generators.

4.1 Validation

To check the generator, the computed dispersion is compared with its expected value using an approximate-equality test that tolerates small decimal differences. The proposed method is compared with random sampling, with the Hopf-coordinate method on S^3 , and with cylindrical mapping on S^4 and S^5 .

The proposed method is markedly more uniform than the Hopf-coordinate method on S^3 , and more uniform than cylindrical mapping on S^4 and S^5 . The advantage is most pronounced when the number of points is small.

5 Conclusions

This paper has presented a construction of low-discrepancy sequences on n -dimensional spheres based on the van der Corput sequence. It addresses the difficulties of high-dimensional sampling while preserving

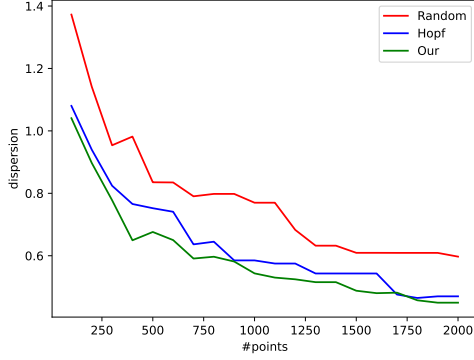


Figure 8: Result for S^3 compared with the Hopf-coordinate method

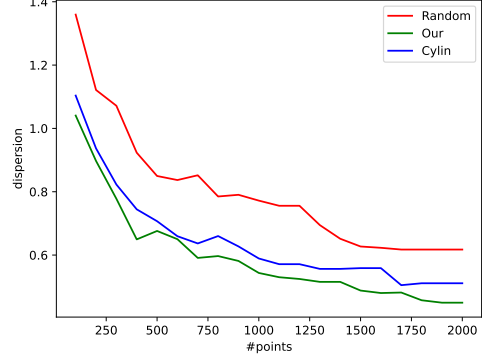


Figure 9: Result for S^3 compared with cylindrical mapping

uniformity, determinism and incrementality, and it covers the van der Corput and Halton sequences, the unit circle and sphere, the Hopf fibration for S^3 , and the recursive extension to S^n .

5.1 Future Work

The method generates point sets of high uniformity efficiently and is simple to implement, which makes it useful for Monte Carlo simulation, optimisation and machine learning. Future work includes faster table lookup or closed-form approximations for large n , and extensions to $SO(n)$ and other manifolds.

Acknowledgements

The authors thank the National Science Foundation for supporting this research.

Author Contributions

The authors contributed equally to this work.

Funding

This research was supported by the National Science Foundation under Grant No. 1234567890.

Competing Interests

The authors declare that they have no competing interests.

Availability of Data and Materials

The data and materials used in this study are available from the corresponding author on request.

References

- Mandic, DP et al. 2011. “Filter Bank Property of Multivariate Empirical Mode Decomposition.” *Signal Processing, IEEE Transactions on* 59 (5): 2421–26.
- Mitchell, Julie C. 2008. “Sampling Rotation Groups by Successive Orthogonal Images.” *SIAM Journal on Scientific Computing* 30 (1): 525–47.
- Rehman, Naveed, and Danilo P Mandic. 2010. “Multivariate Empirical Mode Decomposition.” *Proceedings of the Royal Society A: Mathematical, Physical and Engineering Science* 466 (2117): 1291–302.

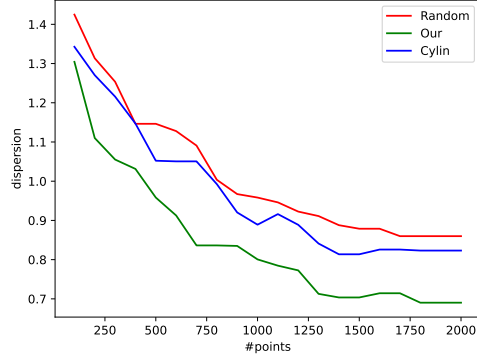


Figure 10: Result for S^4 compared with cylindrical mapping

Utkovski, Zoran, and Juergen Lindner. 2006. “On the Construction of Non-Coherent Space Time Codes from High-Dimensional Spherical Codes.” *Spread Spectrum Techniques and Applications, 2006 IEEE Ninth International Symposium on*, 327–31.

Wong, Tien-Tsin, Wai-Shing Luk, and Pheng-Ann Heng. 1997. “Sampling with Hammersley and Halton Points.” *Journal of Graphics Tools* 2 (2): 9–24.

Yershova, Anna, Swati Jain, Steven M LaValle, and Julie C Mitchell. 2010. “Generating Uniform Incremental Grids on $SO(3)$ Using the Hopf Fibration.” *The International Journal of Robotics Research* 29 (7): 801–12.

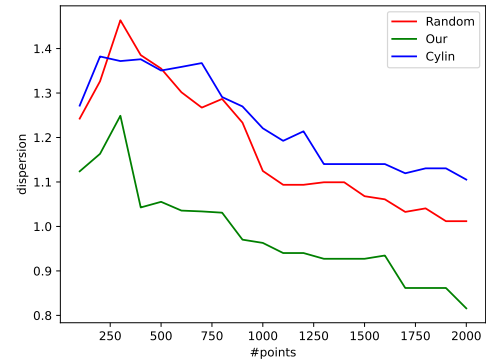


Figure 11: Result for S^5 compared with cylindrical mapping